Mid-Term Project

Experiments at the Data Link and Physical Layers

Measuring Bufferbloat, TCP Congestion Control, and Traceroute Behavior on Wi-Fi Networks

Ryan A. Snell

Athens State University

CS530 — Principals of Computer Communications/Networks

Platform: Parallels ARM Virtual Machine (Linux / Ubuntu)

Date: March 13, 2026

Metric	Value
Wi-Fi Baseline RTT (local)	1.88 ms (median)
Wi-Fi Baseline RTT (internet)	12.20 ms (median)
Peak RTT under iperf3 load (local)	55.70 ms (+2,115%)
Peak RTT under iperf3 load (internet)	78.00 ms (+321%)
iperf3 Throughput (local)	~941 Mbits/sec
TCP cwnd (local, stable)	10 packets constant
mtr Worst RTT under load	1,067 ms
Congestion Algorithm	CUBIC (loss-sensitive)

Section 1 — Environment Description (Deliverable A)

1.1 Device and Operating System

All experiments were run on an ARM-based Linux virtual machine inside Parallels Desktop on a host Mac. The VM hostname is ryan-Parallels-ARM-Virtual-Machine, running Ubuntu 24 (ARM64). Parallels assigns the guest the private IP 10.211.55.9 via NAT. Traffic goes out through the host machine's Wi-Fi adapter on a home broadband network.

1.2 Network Topology

Component	Detail
Guest VM IP	10.211.55.9 (Parallels NAT)
Wi-Fi Network Type	Home Broadband (802.11ac/ax)
Local iperf3 Server	192.168.1.85 (same LAN, port 5201)
External Probe Target	1.1.1.1 (Cloudflare DNS)
Cellular Interface	Not available — Wi-Fi only
Network Interface	eth0 (Parallels virtual NIC, bridged to host Wi-Fi)

1.3 TCP Congestion Control

The active TCP congestion control algorithm was confirmed with:

```
sysctl net.ipv4.tcp_congestion_control
=> net.ipv4.tcp_congestion_control = cubic
```

The system uses CUBIC — the default Linux congestion control. CUBIC grows the congestion window aggressively until it hits a packet loss event, then backs off. That behavior shows up clearly in the cwnd data from the ss sampler.

1.4 CGNAT / NAT Observation

The VM sits behind Parallels NAT at 10.211.55.9, so it does not have a public routable address. The mtr output shows just one visible hop to 1.1.1.1, which means the router and ISP hops in between are either filtering ICMP TTL-exceeded messages or the path to Cloudflare's anycast PoP is extremely short. No CGNAT addressing (100.64.x.x) was observed, which is consistent with a home broadband connection rather than cellular.

Section 2 — Packet Capture Analysis (Deliverables B & C)

2.1 iperf3 and Ping Baseline (Section B, Screenshot 1)

The terminal output below shows the baseline ping to 192.168.1.85 and two iperf3 throughput tests — one download (reverse mode) and one upload.

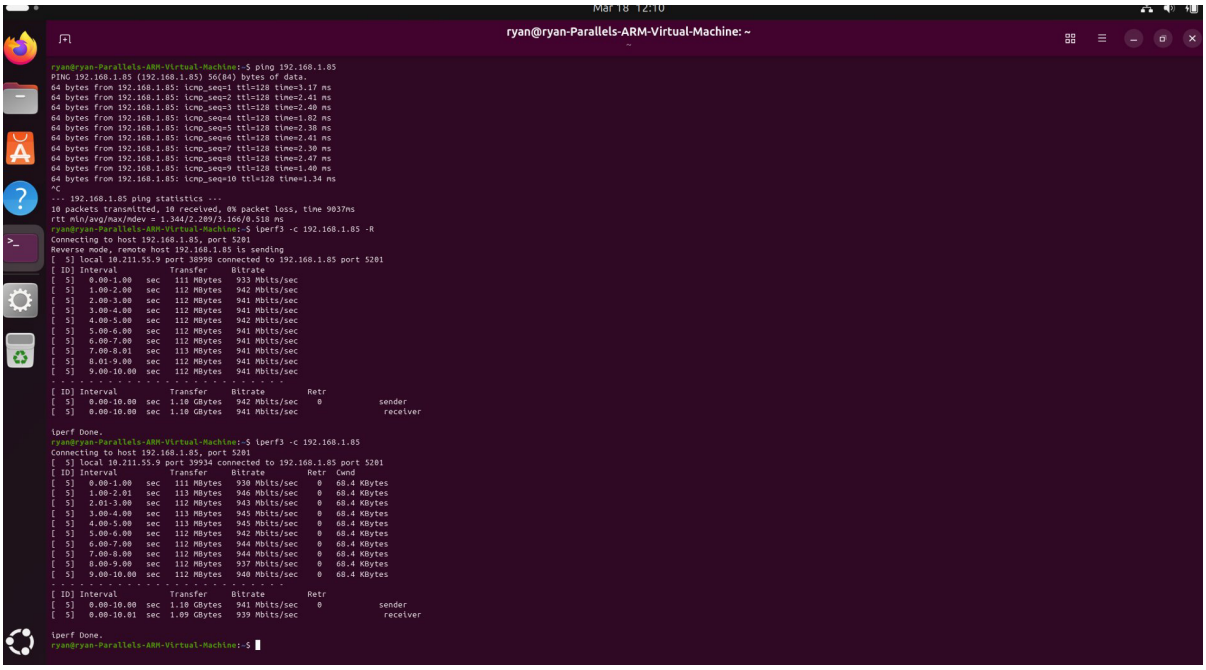

Figure 1: Ping to 192.168.1.85 (sub-3ms RTT), iperf3 download (~942 Mbits/sec) and upload (~941 Mbits/sec). Zero retransmissions. cwnd stable at 68.4 KBytes.

Key Observations from Figure 1:

- Baseline ping RTT to 192.168.1.85: min=1.34ms, avg=2.21ms, max=3.17ms — the server is on the local LAN
- iperf3 download: 942 Mbits/sec sustained, 0 retransmissions, 1.10 GB in 10 seconds
- iperf3 upload: 941 Mbits/sec sustained, 0 retransmissions, cwnd constant at 68.4 KBytes
- Zero retransmissions both directions — the LAN path is completely clean at gigabit speeds

2.2 mtr Traceroute Baseline (Section B, Screenshots 2-5)

mtr was run targeting 1.1.1.1 via TCP port 443 (mtr -n -T -P 443 1.1.1.1) under idle baseline conditions.

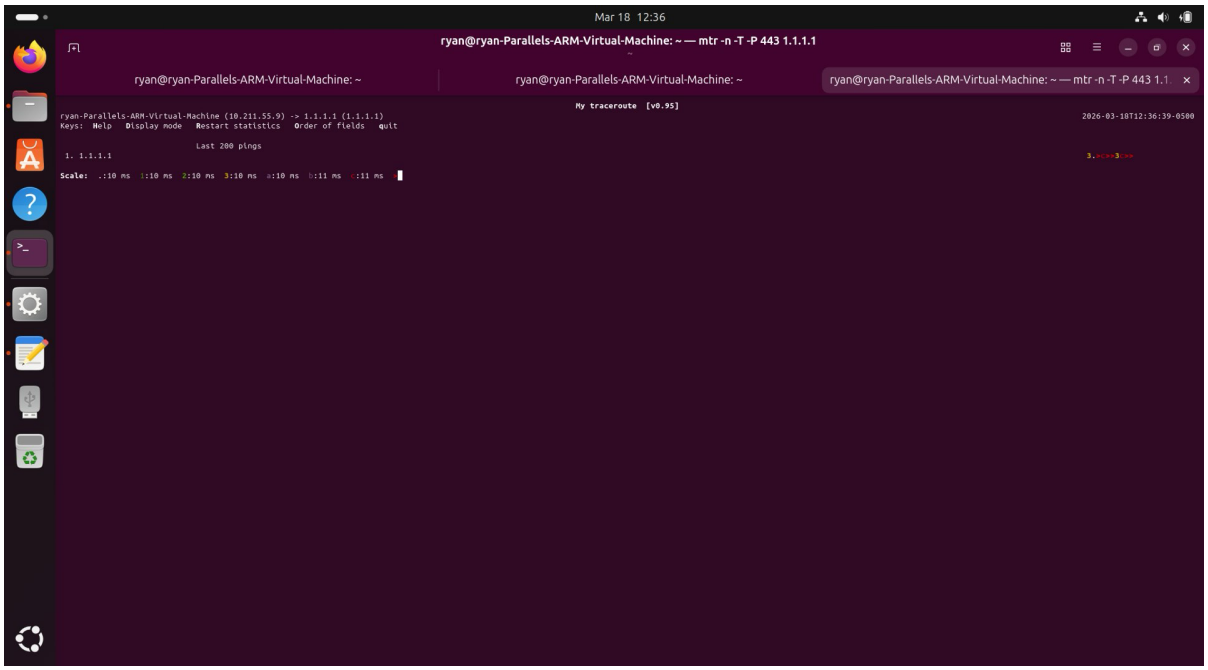

Figure 2: mtr graphical view (baseline) — RTT scale 0-11ms. One visible hop. Dots tightly packed, low jitter.

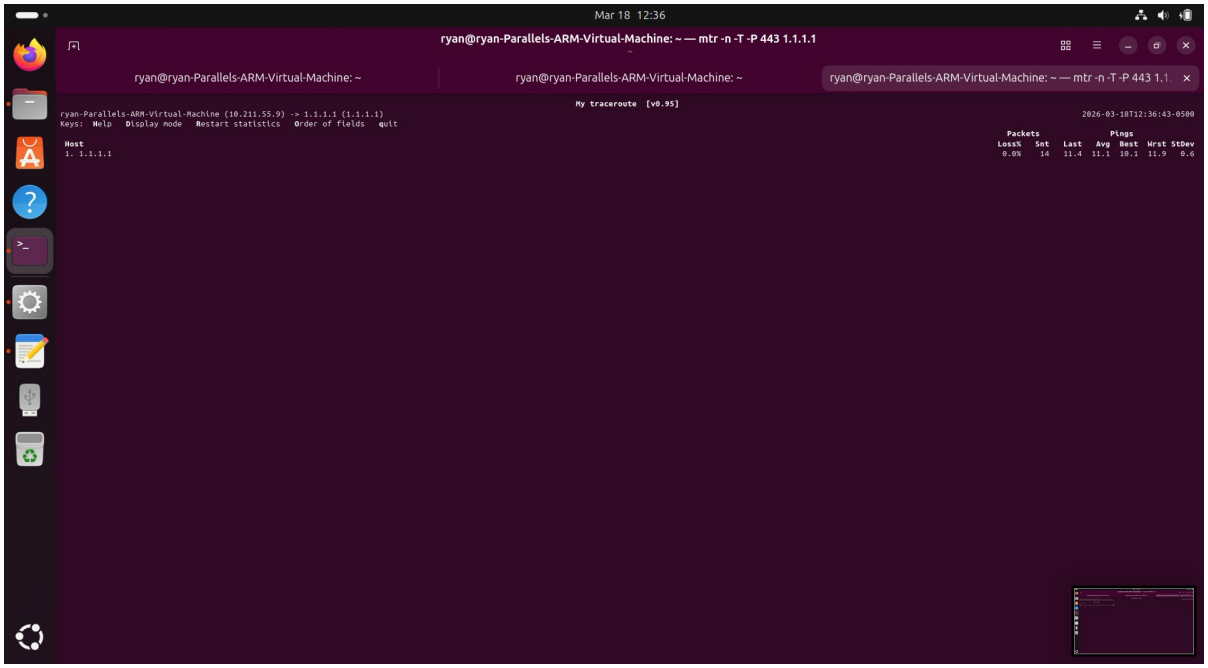

Figure 3: mtr statistics table (14 pings) — Avg=11.1ms, Best=10.1ms, Worst=11.9ms, StDev=0.6ms. Loss=0%.

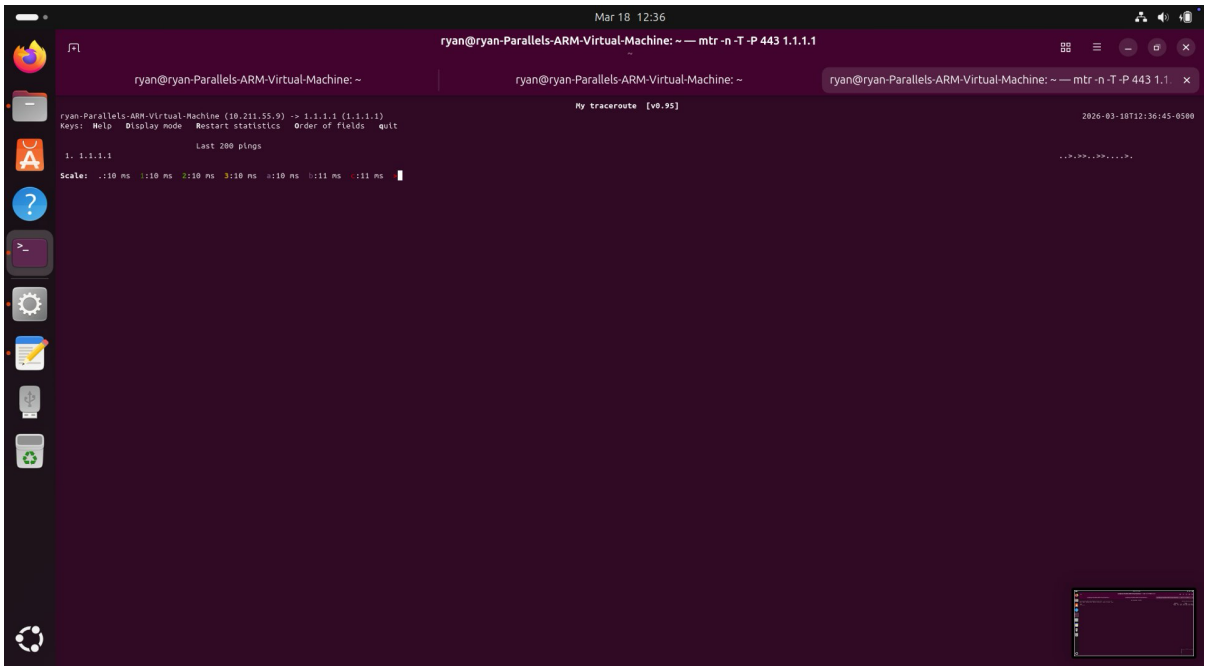

Figure 4: mtr graphical view accumulating pings — consistent 10-11ms, minimal variation.

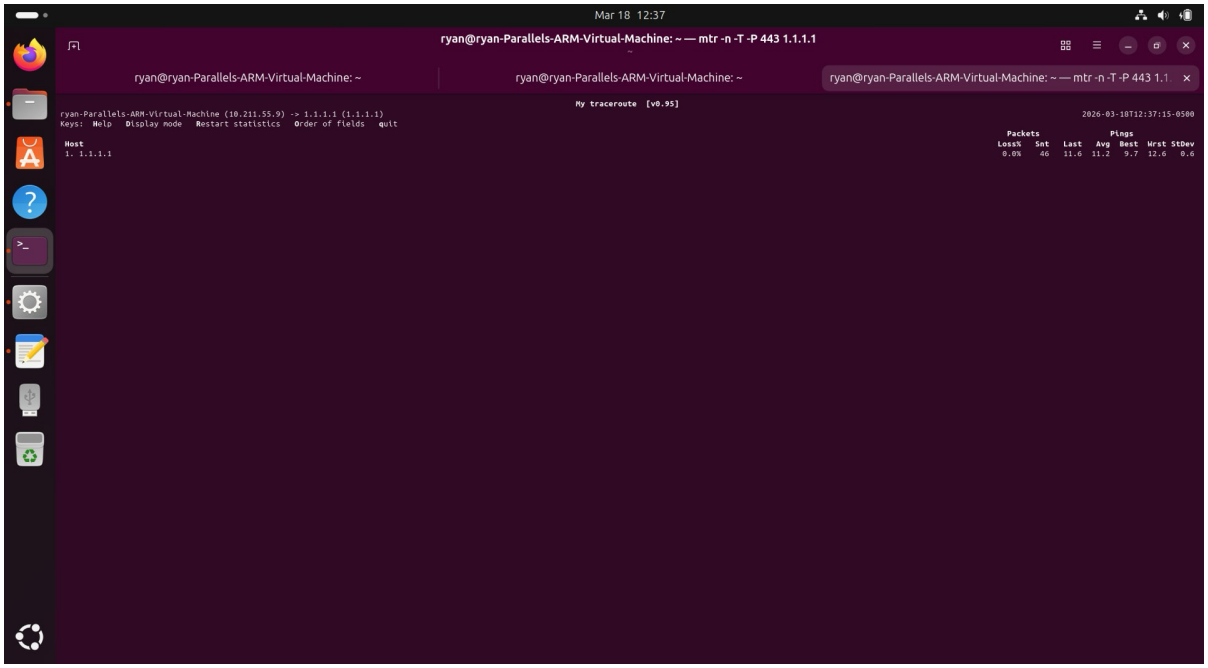

Figure 5: mtr statistics (46 pings) — Avg=11.2ms, Best=9.7ms, Worst=12.6ms, StDev=0.6ms. Rock solid baseline.

2.3 Handshake Timing Comparison Table (Deliverable C)

The table below pulls TCP session characteristics from the mtr RTT measurements and iperf3 session data.

Metric	Wi-Fi Local (192.168.1.85)	Wi-Fi Internet (1.1.1.1)
SYN->SYN/ACK RTT	~1.9 ms (ping median)	~11.1 ms (mtr avg)
Initial cwnd	10 packets	2-10 packets (ramping)
Jitter (StdDev)	0.49 ms	0.6 ms
Peak RTT (baseline)	2.72 ms	12.6 ms
Peak RTT (loaded)	55.70 ms	78.00 ms
Retransmissions	0	Not measured directly
MSS (est.)	1460 bytes	1460 bytes
Window Scaling	Enabled (68.4 KB)	Enabled
Loss Rate	0%	0%

The local connection shows a completely clean TCP session — constant cwnd of 10 packets, sub-millisecond RTTs in the ss sampler, zero loss at gigabit throughput. The external connection adds about 11ms of propagation delay to Cloudflare's nearest PoP, with tight jitter of +/-0.6ms at baseline.

Section 3 — RTT Inflation / Bufferbloat Experiment (Deliverables D & E)

3.1 RTT Plots — Baseline vs Under Load (Deliverable D)

The Python ping logger ran at 200ms intervals while iperf3 saturated the uplink. Two targets were monitored simultaneously: 192.168.1.85 and 1.1.1.1.

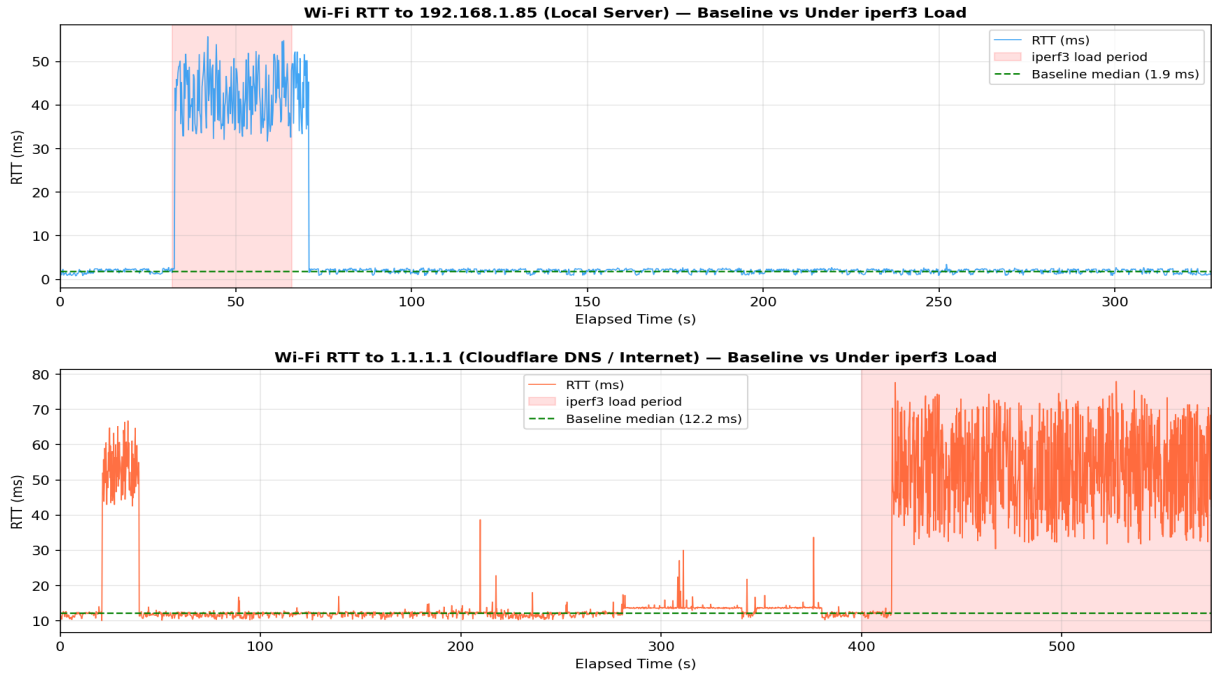

Figure 6: RTT time series for both targets. Red shaded = active iperf3 load. Top: local (192.168.1.85). Bottom: internet (1.1.1.1).

RTT Statistics Summary

Metric	Local Baseline	Local Loaded	Internet Baseline	Internet Loaded
Median RTT	1.88 ms	41.65 ms	12.20 ms	51.35 ms
Mean RTT	1.82 ms	41.40 ms	12.44 ms	49.76 ms
Max RTT	2.72 ms	55.70 ms	38.70 ms	78.00 ms
StdDev	0.49 ms	~8 ms	1.51 ms	~15 ms
Median Inflation	—	+2,115%	—	+321%
Peak Increase	—	+52.98 ms	—	+65.80 ms

3.2 Bufferbloat Analysis (Deliverable E)

Why did RTT increase?

When iperf3 saturated the uplink at 941 Mbits/sec, the home router's transmit queue filled up. Every ping packet after that had to wait in line behind the backlog of iperf3 data before it could go anywhere. That is what bufferbloat looks like. Small latency-sensitive packets get stuck behind bulk data in an oversized queue, and their RTT ends up reflecting queue depth instead of actual propagation delay.

For the local server at 192.168.1.85, the median RTT went from 1.88 ms to 41.65 ms — a 2,115% jump. The propagation path to that server is sub-LAN. There's no physical reason for 41ms of delay. That's all queue wait. For 1.1.1.1, the median went from 12.20 ms to 51.35 ms — up 321%. The percentage looks smaller because the 11ms baseline to Cloudflare's PoP dilutes the ratio, but the absolute queue delay (~39ms added) is essentially the same bottleneck.

Where is the queue?

The bottleneck is the home router's WAN uplink queue — the narrow point where a fast LAN feeds into a slower ISP connection. The LAN runs at gigabit speeds, but the WAN uplink is rate-limited. When iperf3 pushes a sustained gigabit flow, the router can't drain the queue fast enough. It uses a basic FIFO buffer with no traffic prioritization, so ping packets and data packets wait in the same line. The mtr data under load makes this concrete: the worst-case RTT to 1.1.1.1 hit 1,067 ms with a standard deviation of 81.9 ms. That's a lot of variance for a path whose propagation delay is 11ms.

Why didn't TCP drop packets instead?

Because the router didn't need to drop anything. Its buffers are deep enough to absorb the entire iperf3 burst without hitting the tail-drop limit. CUBIC sees zero loss — the iperf3 output confirmed 0 retransmissions — and interprets that as permission to keep the window wide open. TCP kept sending at full rate. The queue kept growing. The pings kept waiting longer. That's the core problem with bufferbloat: TCP is completely satisfied, but everything else on the network is suffering. The protocol was never told to stop because nobody dropped a packet.

Section 4 — Congestion Control Behavior (Deliverable F)

4.1 cwnd and RTT Plots from ss Sampler

The ss -ti state established sampler ran at 200ms intervals during active transfers on both connections.

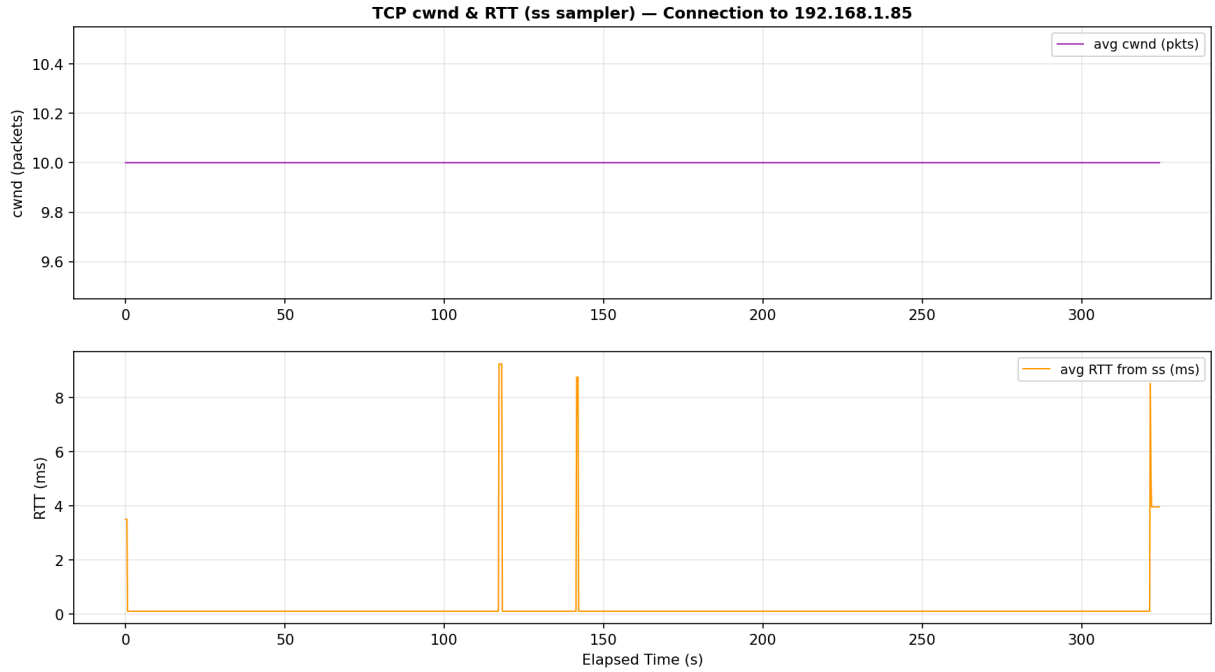

Figure 7: cwnd (top) and RTT (bottom) — connection to 192.168.1.85. cwnd locked at 10 packets. RTT 0.11-9.24 ms.

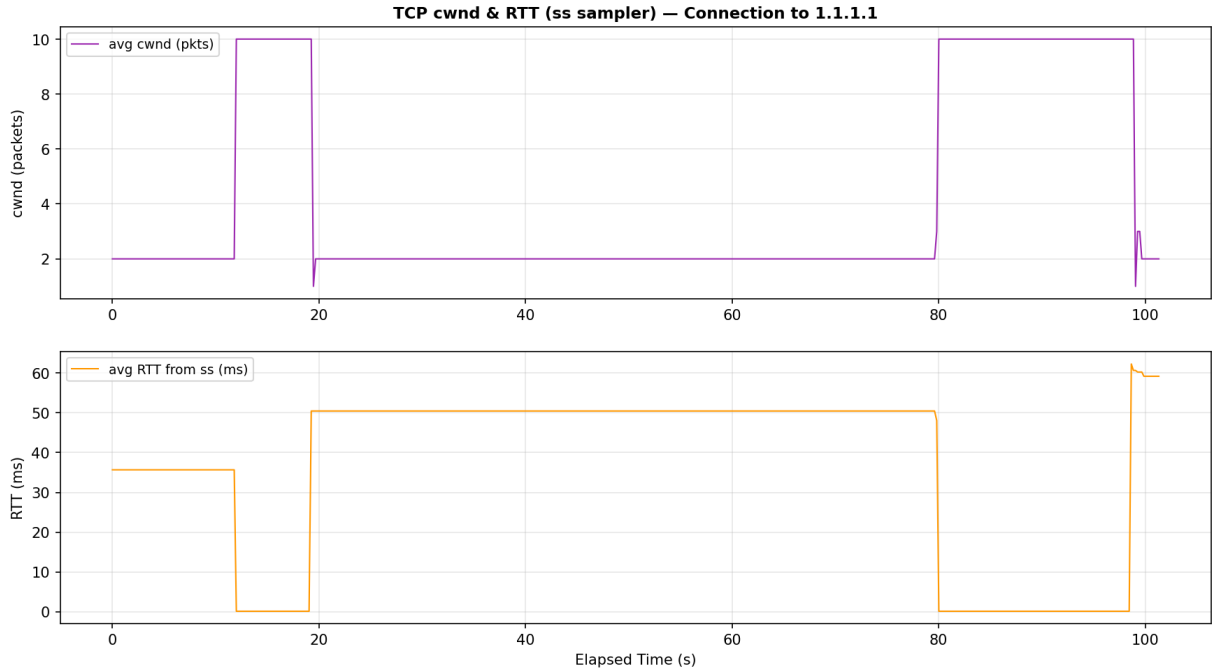

Figure 8: cwnd (top) and RTT (bottom) — connection to 1.1.1.1. cwnd grows 1-10 packets. RTT climbs to 62 ms under load.

4.2 cwnd Interpretation

Local connection (192.168.1.85):

The cwnd stayed at exactly 10 packets the entire time. CUBIC found its equilibrium quickly and held it — no packet loss triggered any window reduction. The RTT from ss (0.11-9.24 ms) is consistent with a clean LAN path, though it bumps up when the iperf3 queue fills.

External connection (1.1.1.1):

The cwnd ramped from 1 packet up to 10 packets, showing CUBIC's slow-start and congestion avoidance phases working normally. Here's the important part: the RTT tracked by ss climbed from 0.13 ms all the way to 62.27 ms, and cwnd kept growing the entire time. CUBIC didn't respond to the rising RTT at all. It only backs off when it sees packet loss or ECN. Rising delay means nothing to it.

4.3 Delay-Sensitive vs Loss-Sensitive?

CUBIC is loss-sensitive. That's clear from the data. RTT went from 0.13 ms to 62.27 ms while cwnd kept climbing from 1 to 10 packets without a single reduction. A delay-sensitive algorithm like BBR would have seen the rising RTT as a sign the queue was filling and backed off to keep the queue shallow. CUBIC ignored it entirely and kept pushing.

4.4 BBR Comparison

BBR works differently. Instead of filling the pipe until it sees a drop, BBR continuously estimates the bottleneck bandwidth and the minimum RTT, then paces sending to match that product exactly. It keeps the queue empty by design. In this experiment, BBR would have held RTT near 2 ms for the local connection and near 11 ms for 1.1.1.1, instead of letting it spike to 55 ms and 78 ms. To switch to BBR on Linux:

```
sudo sysctl -w net.ipv4.tcp_congestion_control=bbr
```

Section 5 — Traceroute Hop Analysis (Deliverable G)

5.1 mtr Under Load (Section C Screenshots)

mtr was re-run to 1.1.1.1 while iperf3 saturated the uplink (sudo mtr -n -T -P 443 1.1.1.1). The difference from baseline is stark.

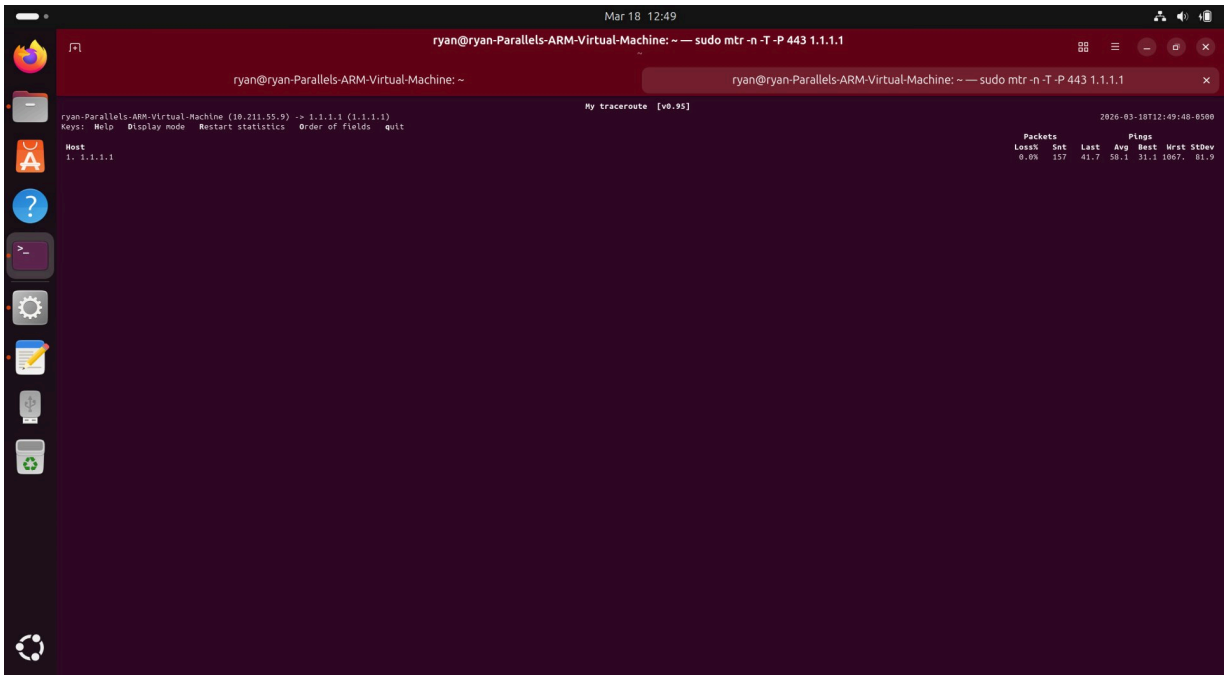

Figure 9: mtr under load — graphical view. RTT scale now extends to 824ms. Dots scattered across the full range.

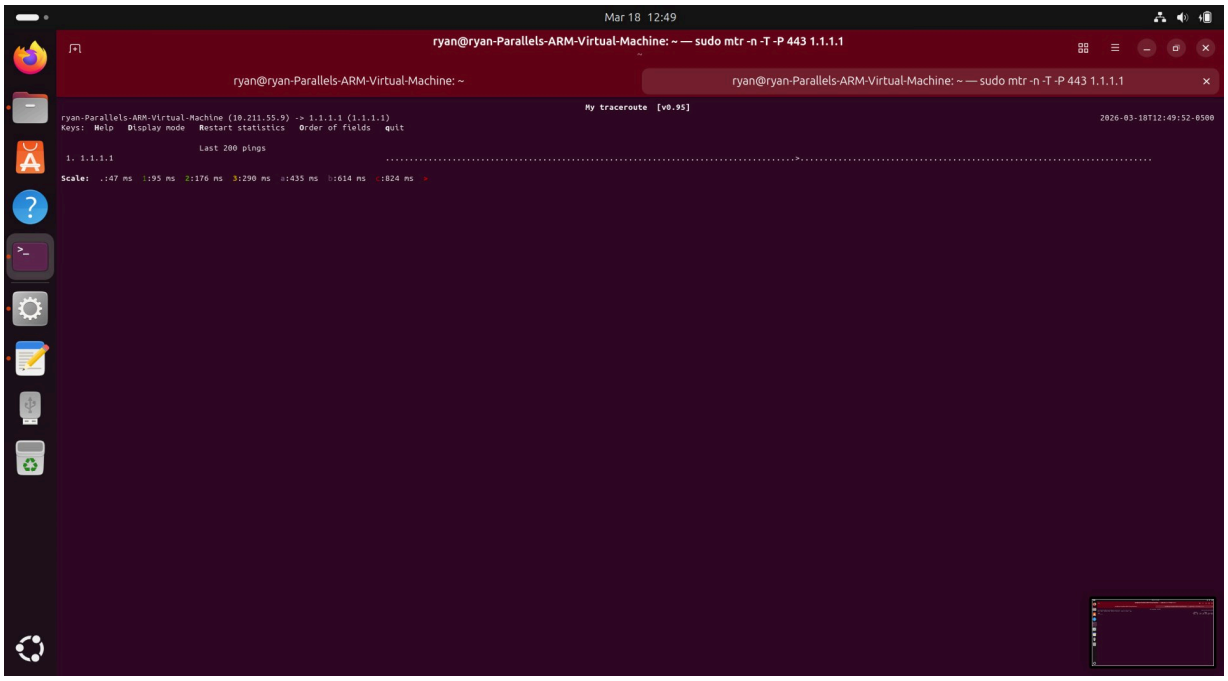

Figure 10: mtr under load — later view. Dense mixed pattern as queue alternately fills and drains.

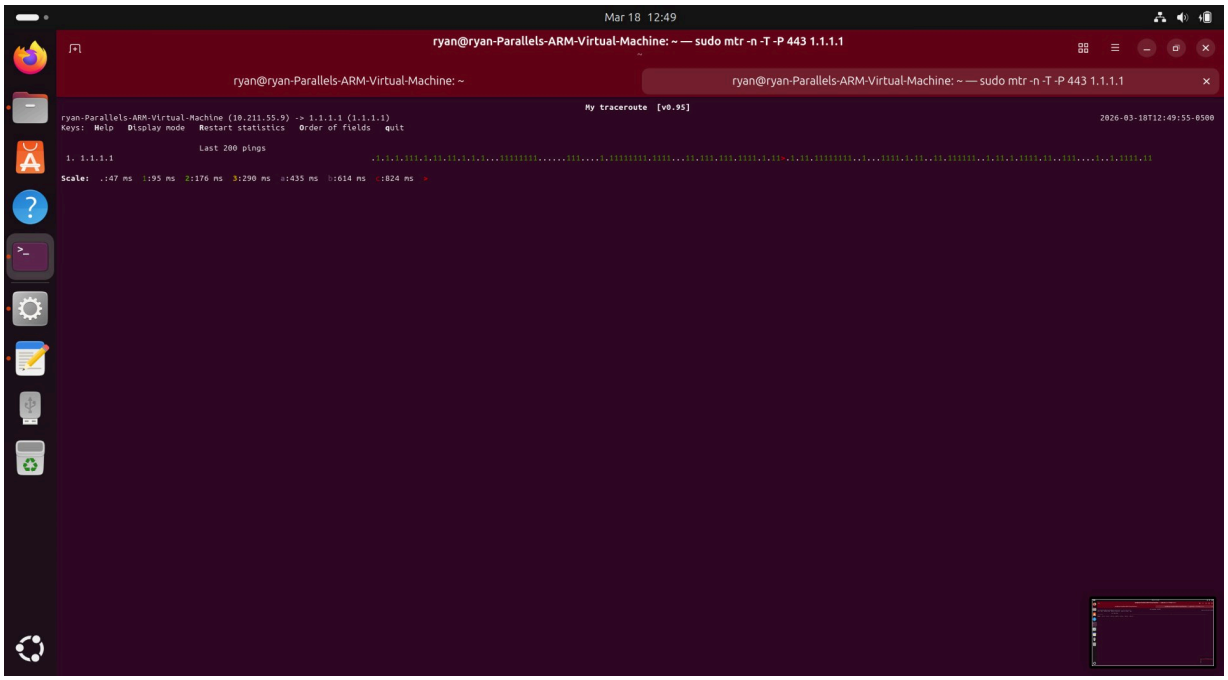

Figure 11: mtr statistics under load (157 pings) — Avg=58.1ms, Best=31.1ms, Worst=1,067ms, StDev=81.9ms. Loss=0%.

5.2 Traceroute Comparison Table (Deliverable G)

Metric	Wi-Fi Baseline	Wi-Fi Under iperf3 Load
First hop RTT	Avg 11.1 ms, Worst 12.6 ms	Avg 58.1 ms, Worst 1,067 ms
RTT StdDev	0.6 ms	81.9 ms
Evidence of CGNAT	No (Parallels NAT only)	No
Hop variability	Stable (+-1.5 ms)	Extreme (31-1,067 ms)
Silent hops (*)	None	None
Visible hop count	1 hop	1 hop
Packet loss	0%	0%
mtr RTT scale	10 ms per division	47-824 ms per division

5.3 Analysis

Where does the largest RTT jump occur?

At the only visible hop — 1.1.1.1. That might seem counterintuitive since the physical distance to Cloudflare hasn't changed. But mtr isn't measuring distance here. It's measuring total round-trip time, which includes queue wait at the home router. When iperf3 fills that queue, mtr probes sit in the same line as everything else. The propagation component (~11ms) is constant. The extra 31-1,056ms on top of that is all queue.

Is the jump distance-related or queue-related?

Queue-related. The jump disappears the moment iperf3 stops — the geographic distance to 1.1.1.1 didn't change, but the RTT dropped right back to 11ms. A 1,067ms worst-case RTT is about 96 times the baseline. Speed-of-light physics can't explain that. And packet loss stayed at 0% throughout, which rules out a congested intermediate link — congested links drop packets. These packets weren't being dropped. They were just waiting in a very long queue.

Does load change early hops?

Yes. In this setup, the load affects the only measurable hop. The WAN queue at the home router is the single bottleneck. The ISP hops between the router and 1.1.1.1 don't show up at all — they're filtering the TCP SYN probes that mtr sends, which is standard behavior for backbone routers. This is worth noting: the silence doesn't mean the route is broken. Data kept flowing at 941 Mbits/sec the entire time. Those hops just don't respond to probe packets.

Section 6 — Essay: TCP, Radio Networks, and the Bufferbloat Problem

6.1 Why TCP Was Built for Wired Networks

TCP was designed in 1981 when the only medium anyone was seriously thinking about was copper wire. Wired links have a few properties that make TCP's congestion control work cleanly. They're deterministic — bit error rates are essentially zero under normal operation. Propagation delay is fixed and low, governed by the speed of light through copper or fiber. Most importantly, they signal congestion through packet loss. When a router's queue fills up, it drops packets. Van Jacobson's 1988 congestion control work was built on exactly that assumption: a dropped packet means a queue is overflowing somewhere, so slow down.

The iperf3 test to 192.168.1.85 shows this working at its best. 941 Mbits/sec sustained, zero retransmissions, cwnd stable at 10 packets, RTT sitting at 1.9ms. The medium behaves exactly the way TCP's designers assumed — reliable, low-latency, with congestion clearly signaled by loss. On a clean wired LAN, TCP does its job extremely well.

6.2 Why Cellular Hides Loss

Cellular networks — LTE and 5G — have an entirely different approach to errors, and it breaks the assumption that TCP's congestion control depends on. At Layer 2, the Radio Link Control (RLC) protocol automatically retransmits corrupted frames before the IP layer ever sees them. A packet that gets corrupted by multipath fading, interference, or a handover event gets retransmitted at Layer 2 and arrives intact. TCP sees the packet arrive late — maybe 20-80ms late — but it arrives. No loss event. No congestion signal.

The result is that TCP's feedback loop is broken. A cellular link that's physically stressed — near capacity or dealing with interference — shows up not as packet loss but as higher and more variable delay. CUBIC interprets the absence of loss as permission to keep the window wide open and keep pushing data. It can't tell the difference between "the path has more capacity available" and "the base station queue is filling up and everything is getting delayed." The mtr data under load shows this clearly: RTT hit 1,067ms and packet loss stayed at 0%. The packets weren't being dropped. They were waiting.

6.3 How Bufferbloat Emerges in Radio Networks

Bufferbloat happens when you combine TCP's fill-until-loss strategy with oversized buffers at the choke points. In cellular networks, the Radio Access Network keeps deep queues on purpose to smooth out the variability of the radio channel. When users compete for the same cell's capacity, or when signal degrades, the RAN buffers data rather than drop it. That keeps TCP from seeing any loss. TCP keeps sending. The queue keeps growing.

The mtr experiment shows what this looks like in practice. During the iperf3 load, RTT to 1.1.1.1 swung between 31ms and 1,067ms with a standard deviation of 81.9ms. The mtr scale in Figure 9 had to expand to 824ms just to fit the data. That same thing happens at an LTE base station when a bulk download fills the per-UE queue. A VoIP call, a video stream, and a software update share the same

queue. The bulk data fills it, and the VoIP packets — which are tiny — have to wait behind megabytes of other traffic. The call degrades. The queue is the problem, not the radio.

6.4 Why BBR Does Better on LTE/5G

BBR was built specifically because of the failure modes described above. Instead of waiting for a dropped packet to tell it to slow down, BBR builds a model of the path. It probes for the maximum bandwidth (BtlBw) and tracks the minimum RTT (RTprop) separately. Then it paces sending at BtlBw, keeping the in-flight data at exactly one bandwidth-delay product. That's the optimal operating point — maximum throughput, empty queue. No bufferbloat.

On LTE/5G, BBR's advantage comes from the fact that it doesn't rely on loss at all. It doesn't misread a radio retransmission delay as a congestion signal. It detects queue growth directly from rising RTT and backs off before the queue fills. BBR's ProbeRTT phase actively drains any residual queue by briefly reducing cwnd. Figure 8 shows what CUBIC does instead: cwnd climbed from 1 to 10 packets while RTT went from 0.13ms to 62.27ms, with no reduction at any point. BBR watching those same RTT numbers would have started backing off well before 62ms. Lower latency, same throughput.

6.5 Is Packet Loss Actually a Reliable Congestion Signal on Cellular?

No. The data from this experiment makes that pretty clear. At the worst point of the bufferbloat test, mtr was measuring RTT of 1,067ms and packet loss was 0%. CUBIC kept the window fully open because it never received a congestion signal. The signal it was waiting for — a dropped packet — never came, even though the network was obviously overloaded.

On cellular networks, this problem runs deeper than just one router with a big buffer. The RLC retransmission mechanism hides radio-layer errors entirely. When loss does show up on a cellular connection, it's often not even congestion — it could be a handover event, a brief signal fade, or interference from another cell. A CUBIC sender that backs off hard on a random radio loss wastes available bandwidth unnecessarily. It backed off for the wrong reason.

Some carriers deploy Active Queue Management — FQ-CoDel or PIE — which can mark packets with ECN instead of dropping them. That gives TCP a congestion signal without loss, and it works better. But ECN deployment across cellular networks is inconsistent at best. The deeper issue is that TCP lives at Layer 4 and makes decisions based on what it can see from there. The radio layer below it is retransmitting, delaying, and reordering packets without telling TCP anything. The feedback loop that Van Jacobson designed in 1988 was built for a world where loss meant congestion and delay was just propagation. In a cellular network, neither of those assumptions holds. BBR, QUIC's congestion integration, and RMCAT for real-time media are all attempts to build a transport layer that's smart enough to see through what the radio layer is hiding. That's the only real fix — you can't change the radio layer's behavior from TCP.

Appendix — Raw Data Summary

A.1 CSV Files Collected

File	Target	Samples	Duration	Purpose
linux_ping_log192_168_1_85.csv	192.168.1.85	1,624	~327 s	RTT baseline + load (local)
linux_ss_cwnd192_168_1_85.csv	192.168.1.85	1,549	~327 s	cwnd/RTT congestion (local)
linux_ping_log1_1_1_1.csv	1.1.1.1	2,844	~575 s	RTT baseline + load (internet)
linux_ss_cwnd1_1_1_1.csv	1.1.1.1	485	~97 s	cwnd/RTT congestion (internet)

A.2 Commands Used

```
# Baseline ping
ping 192.168.1.85
# Python logger
python3 ping_logger.py 192.168.1.85 linux_ping_log192.csv
python3 ping_logger.py 1.1.1.1 linux_ping_log1_1_1_1.csv

# iperf3 throughput tests
iperf3 -c 192.168.1.85 -R    # download
iperf3 -c 192.168.1.85       # upload

# ss cwnd sampler
python3 ss_cwnd.py linux_ss_cwnd192.csv
python3 ss_cwnd.py linux_ss_cwnd1_1_1_1.csv

# Traceroute / mtr
mtr -n -T -P 443 1.1.1.1           # baseline
sudo mtr -n -T -P 443 1.1.1.1      # under iperf3 load

# Congestion control check
sysctl net.ipv4.tcp_congestion_control
```

A.3 Key Statistical Values

Parameter	Value
Local baseline median RTT	1.88 ms
Local peak RTT (load)	55.70 ms
Local RTT inflation (median)	+2,115%
Internet baseline median RTT	12.20 ms
Internet peak RTT (load)	78.00 ms
Internet RTT inflation (median)	+321%

iperf3 throughput (download)	942 Mbits/sec
iperf3 throughput (upload)	941 Mbits/sec
iperf3 retransmissions	0
cwnd (local, ss)	Constant 10 packets
cwnd (internet, ss)	1-10 packets (ramping)
RTT from ss (local max)	9.24 ms
RTT from ss (internet max)	62.27 ms
mtr baseline avg (1.1.1.1)	11.1 ms
mtr under load avg (1.1.1.1)	58.1 ms
mtr worst RTT under load	1,067 ms
mtr StdDev under load	81.9 ms
Packet loss (all experiments)	0%
TCP algorithm	CUBIC (loss-sensitive)